

Plan de projet : Tchip ! Suivi de dépenses Mobile Money

23 septembre 2026 · Christiany AKOTENOU

Objectif

Livrer en 5 semaines « Tchip ! », une app Android qui lit les SMS Mobile Money, classe les dépenses et aide à rester sous un plafond de dépenses en FCFA sur la période choisie.

- **Problème** : les transactions MoMo sont dispersées dans les SMS, impossible de savoir où part l'argent.
- **Premiers utilisateurs** : moi, puis 5 à 10 proches (étudiants et famille).
- **Double but** : m'aider à mieux gérer mes dépenses et avoir un projet concret et utilisé sur mon CV.

Périmètre du MVP

La v1 couvre la collecte automatique, la catégorisation et le budget. Le reste attend la v2.

Inclus en v1	Exclu de la v1
Import de l'historique SMS (MTN, Moov, Celtiis)	Version iOS
Détection des nouveaux SMS en temps réel	Synchronisation cloud et compte utilisateur
Catégorisation automatique + correction manuelle	Comptes bancaires et cartes
Ajout manuel des dépenses en espèces	Objectifs d'épargne
Plafond global et par catégorie, sur une période choisie, avec alertes et passage au rouge	Export PDF ou Excel
Tableau de bord et bilans sur la période choisie (semaine, mois, année, dates libres)	Publication sur le Play Store
Plans sur la période choisie (un jour, une semaine, des dates libres) avec des plafonds par catégorie, par jour ou sur toute la période	Report du reste d'un jour sur le lendemain et modèles de plans
Frais débités par l'opérateur sur chaque transaction (retrait, envoi, paiement) et total des frais sur la période	Comparaison des frais entre opérateurs
Interface en français et en anglais : langue de l'appareil par défaut, modifiable dans les réglages, montants et dates au format de la langue	Autres langues (fon, yoruba)
Interface responsive : petits et grands téléphones, tablettes, mode paysage, grande taille de police système	Version web ou ordinateur
Écran de chargement avec « Tchip ! » et une courte animation dynamique	Onboarding animé en plusieurs étapes

Stack technique

Flutter pour garder un seul code quand iOS arrivera, avec des données 100 % locales.

Besoin	Choix
Framework	Flutter (Dart)
Lecture de l'historique SMS	flutter_sms_inbox
SMS entrants en temps réel	another_telephony
Permissions	permission_handler
Base de données locale	sqflite (SQLite)
Gestion d'état	Riverpod
Graphiques	fl_chart
Notifications d'alerte	flutter_local_notifications
Tests	flutter_test (tests unitaires du parser)
Versionnage	GitHub, avec README et captures
Traduction FR / EN	flutter_localizations + intl (fichiers ARB fr et en)
Responsive	LayoutBuilder et MediaQuery, points de rupture téléphone / tablette
Écran de chargement	flutter_native_splash (écran natif au démarrage) puis animation Flutter (AnimationController, ou Lottie / Rive)

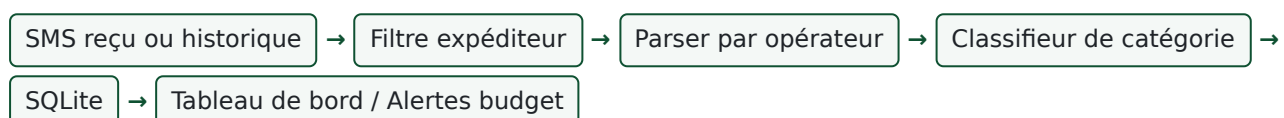
Distribution

La v1 se distribue en APK signé, envoyé directement à mes proches, sans passer par le Play Store.

- **Pendant le dev** : `flutter run` sur mon téléphone en débogage USB ou sans fil.
- **Clé de signature** : un keystore unique (`tchip-release.jks`), gardé hors du dépôt avec `key.properties` dans le `.gitignore`. Perdre la clé oblige à désinstaller l'app pour passer à une nouvelle version.
- **Build** : `flutter build apk --release --split-per-abi`, puis partage de `app-arm64-v8a-release.apk` par WhatsApp ou Drive.
- **Installation** : autoriser les sources inconnues, « Installer quand même » si Play Protect avertit, accepter la permission SMS au premier lancement.
- **Mises à jour** : version augmentée dans `pubspec.yaml` (le numéro après le `+` monte toujours), même clé, nouvel APK installé par-dessus sans perte de données.

Architecture et données

Chaque SMS passe par un parser propre à l'opérateur, puis par le classifieur, avant d'être stocké en local.



La saisie manuelle des dépenses en espèces alimente directement la base SQLite.

Tables principales

Table	Champs clés
transactions	id, date, montant, frais, type (envoi, réception, retrait, paiement, forfait), opérateur, contrepartie, catégorie_id, sms_brut, source (sms ou manuel)
categories	id, nom, icône, couleur
budgets	id, catégorie_id (vide = plafond global), plan_id, plafond, rythme (par jour ou sur toute la période), date (pour un plafond ponctuel sur un seul jour)
plans	id, nom, date_début, date_fin (un jour, une semaine ou des dates libres)
regles	id, mot_clé ou contrepartie, catégorie_id (appries quand l'utilisateur corrige)

Le champ sms_brut permet de rejouer le parsing si les règles changent. Le champ `id` du SMS évite les doublons à l'import.

Planning sur 5 semaines

Début prévu le lundi 28 septembre, fin de la v1 le 1er novembre, à raison d'environ 8 h par semaine.

Semaine 1 (28 sept.) : parser

- ☐ Créer le projet Flutter (package com.tchipmo.app) et le dépôt GitHub tchipmo
- ☐ Collecter 20 SMS réels par opérateur, anonymisés
- ☐ Écrire le modèle Transaction et les parsers MTN, Moov, Celtiis
- ☐ Tests unitaires sur tous les SMS collectés
- ☐ Créer la clé de signature et configurer le build release
- ☐ Extraire les frais de chaque SMS (retrait, envoi, paiement) et tester ce cas pour chaque opérateur
- ☐ Mettre en place la traduction FR / EN (fichiers ARB) et la détection de la langue de l'appareil, avant d'écrire le premier écran

Livable : parser testé à plus de 95 % de réussite

Semaine 2 (5 oct.) : données

- ☐ Permissions SMS et import de l'historique
- ☐ Base SQLite et dédoublonnage
- ☐ Écoute des SMS entrants
- ☐ Écran liste des transactions
- ☐ Premier APK signé installé sur mon téléphone
- ☐ Afficher les frais sous chaque transaction (ex. « Retrait 20 000 F, frais 300 F »)
- ☐ Écran de chargement : « Tchip ! » sur fond sombre, lettres qui apparaissent une à une puis point d'exclamation jaune (#E8B931) qui rebondit, 1,5 s maximum pendant le chargement des données

Livable : mes vraies transactions s'affichent dans l'app

Semaine 3 (12 oct.) : catégories

- ☐ Catégories par défaut et règles par mots-clés
- ☐ Correction manuelle qui crée une règle
- ☐ Formulaire d'ajout de dépense en espèces
- ☐ Widget d'écran d'accueil pour ajouter une dépense en espèces en deux touches
- ☐ Écran Réglages : choix de la langue (automatique, français, anglais)

Livable : 80 % des transactions bien classées sans intervention

Semaine 4 (19 oct.) : budget et tableau de bord

- ☐ Créer un plan sur la période choisie (un jour, une semaine, des dates libres) avec des plafonds par catégorie, par jour ou sur toute la période (ex. nourriture 2 000 F par jour)
- ☐ Notifications à 80 % et 100 % du plafond, puis à chaque nouvelle dépense une fois dans le rouge
- ☐ Tableau de bord en rouge quand le plafond de la période est dépassé
- ☐ Graphiques : répartition du mois et tendance sur 3 mois
- ☐ Écran Bilan avec sélecteur de période : cette semaine, 2 dernières semaines, ce mois, 2 derniers mois, 3 derniers mois, depuis le début de l'année, 12 derniers mois, cette année, dates personnalisées
- ☐ Contenu du bilan : total dépensé et reçu, frais payés (total et détail par type : retrait, envoi, paiement), répartition par catégorie, top des contreparties, comparaison avec la période précédente
- ☐ Écran du jour : dépensé, plafond et reste par catégorie (ex. « Nourriture : 1 200 / 2 000 F, reste 800 F »)
- ☐ Plafond ponctuel pour un jour précis, qui remplace le plafond habituel ce jour-là

Livable : app utilisable au quotidien

Semaine 5 (26 oct.) : test et présentation

- ☐ APK signé partagé à 5 à 10 proches, avec un message d'installation pas à pas
- ☐ Recueil des retours et corrections
- ☐ README, captures d'écran et vidéo démo de 1 min
- ☐ Tests sur plusieurs écrans (petit téléphone, grand téléphone, tablette, paysage) et avec une grande police, dans les deux langues

Livable : v1 publiée sur GitHub avec ses premiers utilisateurs

Risques et parades

Le risque principal est le format des SMS, qui varie selon l'opérateur et peut changer sans prévenir.

Risque	Parade
Formats de SMS variés ou modifiés	Un parser par opérateur, tests sur de vrais SMS, SMS brut conservé pour rejouer
Permission SMS restreinte par Google Play	Distribution de l'APK en direct pour la v1, déclaration Play Store plus tard
Méfiance des proches (données financières)	Aucune donnée envoyée en ligne, le dire clairement dans l'app
Retard à cause des projets Epitech	Semaines 4 et 5 allégeables : graphiques simplifiés d'abord
Catégorisation peu fiable	Règles apprises des corrections avant tout modèle d'IA
Perte de la clé de signature	Keystore et mots de passe sauvegardés en deux endroits, jamais sur GitHub
Proches bloqués à l'installation (sources inconnues, Play Protect)	Guide d'installation court avec captures, envoyé avec l'APK

Critères de réussite

Le projet est réussi si je l'utilise moi-même chaque mois et qu'il a de vrais utilisateurs.

- Parser : plus de 95 % des SMS de transaction reconnus
- Catégorisation : 80 % correcte sans intervention
- Utilisation : au moins 5 proches actifs après 1 mois
- Personnel : un budget mensuel respecté sur au moins 2 catégories
- Langues : aucun texte en dur, 100 % des textes disponibles en français et en anglais
- Responsive : aucun texte coupé ni débordement sur un petit écran (320 dp de large) ou avec la police système à 130 %

Sur le CV, une ligne chiffrée du type : « Tchip ! App Android Flutter de suivi de dépenses Mobile Money, parsing de SMS multi-opérateurs, stockage local, utilisée par X personnes », avec lien GitHub et vidéo démo.

Et après la v1

L'iOS arrive en premier, le reste selon les retours des utilisateurs.

1. iOS (semaine 6) : raccourci Raccourcis à installer + détection du presse-papier, même parser
2. Classifieur par IA pour les transactions sans règle
3. Objectifs d'épargne et suivi des tontines
4. Export PDF mensuel
5. Publication Play Store après déclaration de la permission SMS
6. Firebase App Distribution pour prévenir les testeurs à chaque nouvelle version
7. Report du reste d'un jour sur le lendemain et modèles de plans réutilisables (« Semaine de cours », « Semaine de vacances »)
8. Comparaison des frais entre opérateurs et conseils pour en payer moins